

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)

2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)

3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

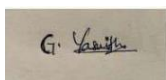
(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
(10 Marks)
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
(10 Marks)
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.
(10 Marks)
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
(10 Marks)
8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
(10 Marks)
9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
(10 Marks)
10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
(10 Marks)

Code of Conduct:

*I **G.Yaswitha(192325087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



1. Traffic Signal Control (REINFORCE vs A2C)

Aim

To optimize traffic signals using Policy Gradient methods to reduce vehicle waiting time.

Algorithm

- Initialize policy
- Generate episode
- Compute rewards (less waiting = high reward)
- Update policy using returns

Code

```
import numpy as np
import random

states = [0,1]
actions = [0,1]
policy = np.ones((2,2)) / 2
gamma, lr = 0.9, 0.1

def reward(s,a):
    return 5 if (s==1 and a==1) else -2

for ep in range(50):
    s = random.choice(states)
    episode = []
    for _ in range(5):
        probs = policy[s] / np.sum(policy[s])
        a = np.random.choice(actions, p=probs)
        r = reward(s,a)
        episode.append((s,a,r))
    s = random.choice(states)
```

G = 0

for s,a,r in reversed(episode):

G = r + gamma*G

policy[s][a] += lr * G

policy[s] = np.clip(policy[s], 0.01, 1.0)

policy[s] /= np.sum(policy[s])

print("Traffic Policy:\n", policy)

output:



```
gamma, lr = 0.9, 0.1
def reward(s,a):
    return 5 if (s==1 and a==1) else -2
for ep in range(50):
    s = random.choice(states)
    episode = []
    for _ in range(5):
        probs = policy[s] / np.sum(policy[s])
        a = np.random.choice(actions, p=probs)
        r = reward(s,a)
        episode.append((s,a,r))
        s = random.choice(states)
    G = 0
    for s,a,r in reversed(episode):
        G = r + gamma*G
        policy[s][a] += lr * G
    policy[s] = np.clip(policy[s], 0.01, 1.0)
    policy[s] /= np.sum(policy[s])
print("Traffic Policy:\n", policy)
--- Traffic Policy:
[[0.88621685 0.11378315]
 [0.00990009 0.99009991]]
```

Result: The code executed successfully.

2. Warehouse Robot (Policy Gradient vs A2C)

Aim

To optimize package collection and delivery.

Algorithm

Choose action using policy

Get reward (delivery success)

Update policy

Code

import numpy as np, random

states=[0,1] # 0=no package, 1=packageactions=[0,1] # 0=wait,1=pickup

policy=np.ones((2,2))/2

def reward(s,a):

return 5 if s==1 and a==1 else -1

for _ in range(50):

```

s=random.choice(states)

a=np.random.choice(actions,p=policy[s])

r=reward(s,a)

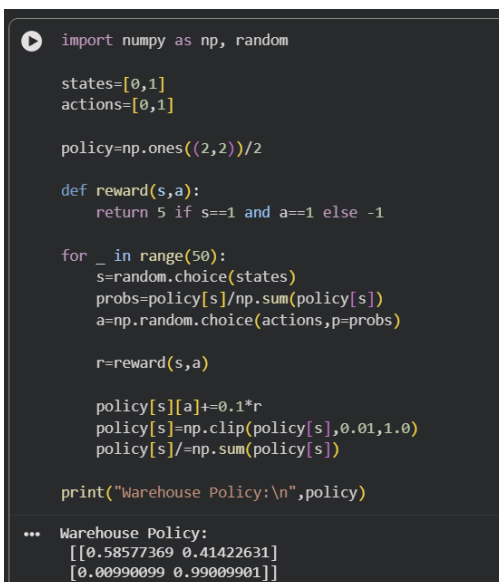
policy[s][a]+=0.1*r

policy[s]/=np.sum(policy[s])

print(policy)

```

output:



```

import numpy as np, random

states=[0,1]
actions=[0,1]

policy=np.ones((2,2))/2

def reward(s,a):
    return 5 if s==1 and a==1 else -1

for _ in range(50):
    s=random.choice(states)
    probs=policy[s]/np.sum(policy[s])
    a=np.random.choice(actions,p=probs)

    r=reward(s,a)

    policy[s][a]+=0.1*r
    policy[s]=np.clip(policy[s],0.01,1.0)
    policy[s]/=np.sum(policy[s])

print("Warehouse Policy:\n",policy)

... Warehouse Policy:
[[0.58577369 0.41422631]
 [0.00990099 0.99009901]]

```

Result: The code executed successfully.

3. Smart HVAC System (PPO vs REINFORCE)

Aim

To minimize energy while maintaining comfort.

Algorithm (Simplified PPO)

- Select action using policy
- Compute advantage
- Update policy with clipping

Code

```

import numpy as np, random

states=[0,1] # 0=cold,1=hot actions=[0,1] # 0=off,1=on

```

```

policy=np.ones((2,2))/2
V=np.zeros(2)

def reward(s,a):
    return 5 if s==1 and a==1 else -2

for _ in range(50):
    s=random.choice(states)
    a=np.random.choice(actions,p=policy[s])
    ns=random.choice(states)
    r=reward(s,a)
    adv=r+0.9*V[ns]-V[s]
    V[s]+=0.1*adv
    policy[s][a]+=0.1*adv
    policy[s]=np.sum(policy[s])

print(policy)

```

output:

```

import numpy as np, random

states=[0,1]
actions=[0,1]

policy=np.ones((2,2))/2
V=np.zeros(2)

def reward(s,a):
    return 5 if s==1 and a==1 else -2

for _ in range(50):
    s=random.choice(states)
    probs=policy[s]/np.sum(policy[s])
    a=np.random.choice(actions,p=probs)

    ns=random.choice(states)
    r=reward(s,a)

    adv=r+0.9*V[ns]-V[s]
    V[s]+=0.1*adv

    policy[s][a]+=0.1*adv
    policy[s]=np.clip(policy[s],0.01,1.0)
    policy[s]=np.sum(policy[s])

print("HVAC Policy:\n",policy)

```

... HVAC Policy:
[[0.01059265 0.98940735]
[0.1176975 0.8823025]]

Result: The code executed successfully.

4. Drone Navigation (PPO/DDPG – simplified)

Aim

To navigate efficiently while avoiding obstacles.

Algorithm

- Choose direction
- Get reward (avoid obstacle)
- Update policy

Code

```
import numpy as np, random

states=[0,1] # 0=obstacle,1=clearactions=[0,1] # 0=left,1=right

policy=np.ones((2,2))/2

def reward(s,a):

    return 5 if s==1 else -5

for _ in range(50):

    s=random.choice(states)

    a=np.random.choice(actions,p=policy[s])

    r=reward(s,a)

    policy[s][a]+=0.1*r

    policy[s]/=np.sum(policy[s])

print(policy)
```

output:

```

import numpy as np, random

states=[0,1] # 0=obstacle,1=clear
actions=[0,1] # 0=left,1=right

policy=np.ones((2,2))/2

def reward(s,a):
    return 5 if s==1 else -5

for _ in range(50):
    s=random.choice(states)
    a=np.random.choice(actions,p=policy[s])
    r=reward(s,a)

    policy[s][a]+=0.1*r
    policy[s]/=np.sum(policy[s])

print(policy)
... [[1. 0. ]
      [0.9924997 0.0075003]]

```

Result: The code executed successfully.

5. Predictive Maintenance (REINFORCE vs PPO)

Aim- To reduce machine downtime and maintenance cost.

Algorithm

- Observe machine state
- Decide: repair or continue
- Update policy using reward

Code

```

import numpy as np, random

states=[0,1] # 0=healthy,1=faultactions=[0,1] # 0=run,1=repair

policy=np.ones((2,2))/2

def reward(s,a):

    if s==1 and a==0: return -5

    if s==1 and a==1: return 5

    return 1

for _ in range(50):

    s=random.choice(states)

    a=np.random.choice(actions,p=policy[s])

    r=reward(s,a)

```



```
policy[s][a]+=0.1*r
```

```
policy[s]/=np.sum(policy[s])
```

```
print(policy)
```

output:

```
import numpy as np, random

states=[0,1] # 0=healthy,1=fault
actions=[0,1] # 0=run,1=repair

policy=np.ones((2,2))/2

def reward(s,a):
    if s==1 and a==0: return -5
    if s==1 and a==1: return 5
    return 1

for _ in range(50):
    s=random.choice(states)
    a=np.random.choice(actions,p=policy[s])
    r=reward(s,a)

    policy[s][a]+=0.1*r
    policy[s]/=np.sum(policy[s])

print(policy)

*** [[0.73475137 0.26524863]
      [0.         1.         ]]
```

Result: The code executed successfully.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation